

SecureVault — Design Report

ENCS4320 Applied Cryptography, Term 1253 — Birzeit University

Abdallah Hattab (1230588) · Abdallah Murra (1230534) · Osama Aabed (1201164)

1. Threat model

1.1 The parties

Users run our client software on machines we assume are trustworthy. Each user has a password known only to them. An attacker cannot read an honest user's memory or disk.

The server stores accounts and documents and relays them between users. It is honest-but-curious: it follows the protocol, neither deletes nor fabricates data, and reads everything it is given. Its entire database may fall into an attacker's hands.

The certificate authority is a third party, separate from the server, that signs user public keys at registration. It is trusted to bind names to keys correctly. Its private key is not available to the server or the attacker. This is an assumption, not a result, and §5.1 discusses what it costs us.

The attacker observes and modifies all network traffic, replays old messages, registers accounts of their own, and may obtain a complete copy of the server's stored data. In our simulated network the attacker's position is even stronger than the specification requires: because client and server communicate through a shared directory, the attacker simply opens the files and edits them. Every attack in attacks/ operates from that position.

1.2 Explicitly out of scope

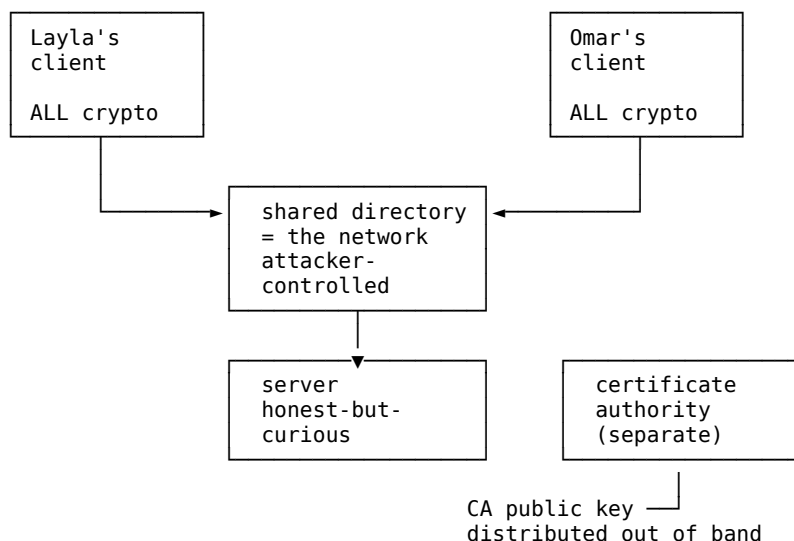
Per Section 4 of the specification: malware on a user's own machine, coercion of a user, traffic analysis of message sizes and timing, and denial of service. We note in §5 where each of these would bite.

One consequence worth stating early: because the user's own machine is outside the attacker's reach, side-channel resistance within the client is not a security requirement of this design. We still took the cheap precautions — constant-time tag comparison, double-and-add-always scalar multiplication — but we do not claim the implementation is hardened, and §5.6 says why it cannot be in Python.

2. Design

2.1 Architecture

Three programs and one shared directory:



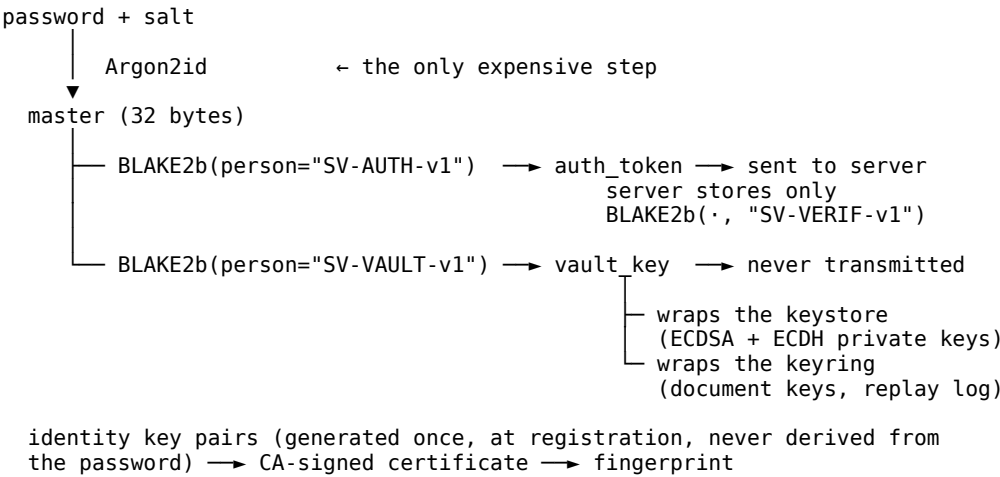
The server is a store and a relay. Every cryptographic decision is made in `client.py`. If that file is correct, the server's honesty is irrelevant.

What the server is structurally prevented from doing. Each item follows from where the keys are, not from the server choosing to behave:

It cannot...	Because...
read a document	document keys are generated on the client and leave it only wrapped under an ECDH-derived key between two users
open a keystore or keyring	both are encrypted under <code>vault_key</code> , which is derived from the password on the client and never transmitted
recover a password	it stores <code>BLAKE2b(auth_token)</code> , and each guess costs a full Argon2id evaluation
forge a certificate	the CA's signing key is in a separate program; the server holds only the CA's public key
alter document metadata undetected	the metadata is the AAD of the document's GCM encryption
alter, redirect or fabricate a share	the manifest inside is signed by the sender and names both parties
tell whether a share is signed, or by whom	the signature is inside the encrypted payload

`server.py` imports no encryption function at all — only `BLAKE2b` and a constant-time comparison. Making the server able to decrypt anything would require adding an import, which a reviewer would notice.

2.2 Key hierarchy



Two separate identity key pairs, one for ECDSA and one for ECDH. Using a single key pair for both signing and key agreement is a known mistake: the two algorithms interact, and a signing oracle can be turned into an attack on the agreement key. The cost is one extra scalar multiplication, once, at registration.

Everything below master is separated by `BLAKE2b` personalisation strings, so a value produced for one role can never be replayed as a value for another. The complete list is in `params.py`.

2.3 Byte-level formats

Seven object types. Full field-by-field tables, with the reasoning for each design choice, are in `securevault/formats.py`; the essentials:

Object	Stored by	Key	Authenticated data
Credential record	server	—	—
Keystore	server	<code>vault_key</code>	header (magic, version, username, nonce)
Keyring	server	<code>vault_key</code>	header
Certificate	server	CA private key (signature)	the whole body incl. username
Document	server	fresh per-document key	the whole metadata header
Manifest	inside a share	sender's ECDSA key (signature)	the whole manifest
Share record	server	ECDH-derived wrapping key	routing header

Two rules shape all of them:

1. **Every field is fixed-width or length-prefixed.** No delimiters. A parser that scans for a separator can be confused by data containing that separator; a parser told the length up front cannot.

2. **Anything covered by a tag or signature is covered as the exact bytes on disk, never as a re-encoding.** Every parse function returns the raw header slice alongside the parsed fields, and the caller authenticates that slice. Re-encoding would mean that any disagreement between our encoder and parser could let two different byte strings authenticate as the same object.

We rejected JSON (no canonical form, so “sign the JSON” is ambiguous) and pickle (executes code on load, which would hand an attacker who controls the server’s storage a way to run code in the client).

2.4 The life of a document

Upload. Generate a fresh 128-bit document key — used for this document and no other, ever. Build the metadata header: id, owner, file name, MIME type, size, timestamp, and a fresh 96-bit nonce. Encrypt with AES-128-GCM under the document key, with the whole metadata header as associated data. Send header, ciphertext and tag to the server. Store the document key in the keyring.

Share (Layla — Omar). Fetch Omar’s certificate; verify the CA’s signature and that it names Omar — this is where a lying server is caught. Build a manifest: manifest id, document id, layla, omar, BLAKE2b(plaintext), BLAKE2b(metadata header), timestamp. **Sign it.** Then ECDH between Layla’s agreement key and Omar’s, hashed with both names and the document id into a wrapping key; **encrypt** manifest || signature || document key under it, with the share’s routing header as associated data.

Retrieve (Omar). Decrypt the share. Verify Layla’s signature over the manifest against the signing key in her certificate. Check the manifest names Omar as recipient, names the sender the routing header claims, carries the document id the header claims, is recent, and has an identifier never accepted before; record the identifier. Fetch the document; check its metadata header hashes to the value Layla signed. Decrypt — GCM verifies ciphertext and metadata together. Check the plaintext hashes to the value Layla signed. Only then return the bytes.

Nothing reaches the caller before the last step, and every check raises the same opaque error.

3. Justification

3.1 Password protection (§7.1)

Chosen: Argon2id, per-user 16-byte salt from `os.urandom`, stored in the clear.

Rejected. PBKDF2 is a chain of HMAC calls costing time but almost no memory, so a GPU’s thousands of cores recover nearly the full parallel speed-up, and raising the iteration count hurts us as much as the attacker. bcrypt’s 4 KiB working set fits in on-chip memory on a GPU or FPGA. scrypt is memory-hard but its access pattern is entirely data-dependent, making it easier to attack through cache timing. Argon2id runs the first half of its first pass with data-**independent** addressing and everything after with data-**dependent** addressing, which is what RFC 9106 recommends when you face both threats — and we do: the server observes our process while it runs *and* its database may be stolen.

The work factor, and an honest problem. Our Argon2id is written from scratch in Python. Measured (`tools/bench.py`):

Profile	m (KiB)	t	p	ours	reference C	ratio
demo	1,024	1	1	0.33 s	0.8 ms	398×
production	65,536	3	4	100.9 s	117 ms	860×

If we had tuned so that *our* verification took 0.25 s, we would have landed on $m = 1$ MiB — which an attacker with compiled Argon2 computes in under a millisecond, about 760 guesses per second per core. The work factor must be argued from the attacker’s side of the table, not ours; our slowness costs us and does nothing to them.

So we define two profiles. **production** ($m = 64$ MiB, $t = 3$, $p = 4$, RFC 9106’s second recommended option) is the design. **demo** exists only to keep the live demonstration responsive and is below RFC 9106’s floor. The profile name is recorded in every credential record, so the stored data itself shows which produced it. Appendix B quantifies the offline attack against both.

Why the salt is not secret. Its job is to make each user’s hash a separate problem, so that one precomputed table cannot attack the whole database and two users with the same password do not produce the same record. Neither requires secrecy. 128 bits makes collisions negligible across any realistic population.

Does anything derived from the password serve a second purpose? Yes, and §2.2 shows how they are kept apart. master yields `auth_token` (sent to the server) and `vault_key` (never transmitted) under different personalisation strings; recovering one from the other means inverting BLAKE2b.

Why the server may store a fast hash of `auth_token`. Argon2id is slow because passwords are low-entropy and guessable. `auth_token` is not a password — it is a uniform 256-bit value with no dictionary to run against it. The fast hash exists only so that a stolen database cannot be replayed directly as a login.

What breaks on a password change? `master`, `auth_token` and `vault_key` all change, so the keystore and keyring are re-wrapped and the server receives a new verifier. The identity key pair is untouched: same public key, same fingerprint, same certificate. Everyone who has verified Layla stays verified, and documents shared before the change still open. This is exactly why the private keys are *wrapped* under a derived key rather than *derived* from the password.

3.2 Document contents (§7.2)

Chosen: AES-128-GCM, implemented as AES-CTR plus GHASH.

Why a dedicated AEAD rather than composing our own. GCM is encrypt-then-MAC by construction — the tag covers the ciphertext, never the plaintext — so a forged ciphertext is rejected before any decryption result is released. Composing AES-CTR with a MAC ourselves would have required choosing that order by hand, and choosing MAC-then-encrypt is the classic padding-oracle mistake. We did not want a security property to depend on our getting an ordering right under deadline. Among AEADs, CCM requires the message length up front and is two passes; GCM is one pass and its GHASH is cheap to implement correctly over $GF(2^{128})$.

Why AES-128 and not AES-256. Our security level is 128 bits throughout: P-256 gives $\approx 2^{128}$ work against Pollard’s rho, and the GCM tag is 128 bits. A 256-bit symmetric key behind a 128-bit-strength curve would not raise the system’s strength — the curve would still be the weakest component — and the specification explicitly warns against mismatched parameter sizes.

How many keys does one document need? One. A fresh 128-bit key per document, generated on the client, which encrypts exactly one message.

How are nonces produced, and what guarantees they never repeat? This is the question the specification calls the single most common way systems like this fail, and our answer is structural rather than procedural. The nonce is 96 random bits, but that is the second line of defence. The first is that *every document gets a fresh key*, so no key is ever used twice with any nonce — after a crash, after a restart, or with two clients running at once. There is no counter to persist and therefore no counter to lose.

`attacks/attack_weak_nonce.py` demonstrates what this buys: against a build with the nonce discipline removed, a confidential document is recovered in full from $C_1 \oplus C_2 \oplus P_1$; against the correct build the same computation yields noise.

3.3 Public-key cryptography (§7.3)

Chosen: ECDH and ECDSA, both on NIST P-256. Digest BLAKE2b-256; deterministic nonces by the RFC 6979 construction with keyed BLAKE2b as the PRF.

Where public-key cryptography is needed at all. Two places, neither solvable symmetrically. Layla and Omar share no secret and their only channel runs through a server that reads everything — only key agreement over a public channel solves this. And Omar must be able to prove authorship to a third party, which a shared-key primitive cannot do (§3.5).

Why P-256. The security level matches everything else at 128 bits. The arithmetic is tractable: p is a Solinas prime and $a = -3$, which admits the cheaper doubling formula. It is fully specified in FIPS 186-4 and SEC 2, so an independent implementation can be checked against published parameters. And signatures and key agreement share one key format.

What we gave up. Curve25519 is more resistant to implementation mistakes — every 32-byte string is a valid public key, so invalid-curve attacks do not arise, and its Montgomery ladder is naturally constant-time. We chose P-256 because it is the curve our course covers. The cost is that we must validate every public key we receive; `p256.validate_public_key()` is that defence and it is not optional. Without it, an attacker could hand us a point on a deliberately weak curve and learn our private key a few bits at a time.

Quantifying the advantage over finite-field DH (`tools/bench_ec.py`). At the 128-bit level, finite-field DH needs a ≈ 3072 -bit modulus against 256 bits for the curve. Comparing wall-clock times would be dishonest — Python’s `pow()` is optimised C while our curve code is interpreted — so we count field multiplications and measure what one costs at each size:

	operations	cost per op	total
P-256 scalar multiplication	$\sim 6,144 \times 256$ -bit mults	1×	6,144
MODP-3072 exponentiation	$\sim 384 \times 3072$ -bit mults	63× (measured)	24,192

Finite-field DH costs $\approx 3.9\times$ the curve at equal security, and public keys are 384 bytes against 65 ($5.9\times$ larger). Our figure is pessimistic on purpose: `scalar_mult` uses double-and-add-always, so it performs a dummy addition on every zero bit. A plain double-and-add would need $\sim 4,096$ multiplications, putting the ratio at $5.9\times$. We report the number our actual code achieves, not the better one we could have had by dropping the side-channel defence.

Why deterministic nonces. ECDSA’s per-signature nonce k is the most dangerous value in the system. Repeat k across two signatures under one key and the private key falls out with school algebra:

$s_1 = k^{-1}(z_1 + rd)$, $s_2 = k^{-1}(z_2 + rd)$ with the same $r \implies k = (z_1 - z_2)/(s_1 - s_2)$, then $d = (s_1k - z_1)/r$

This is how the PlayStation 3 signing key was recovered in 2010 and how Android wallet keys were stolen in 2013. Even slight bias is fatal: lattice attacks recover the key from a few hundred signatures with a handful of biased bits. RFC 6979 removes randomness entirely — k is a PRF of the private key and the message.

Our documented deviation. RFC 6979 instantiates its PRF with HMAC. We use keyed BLAKE2b with personalisation `SV-6979-PRF`. BLAKE2b is designed to be keyed directly and is a PRF on its own, so the HMAC nesting that exists to repair Merkle–Damgård hashes buys nothing here. To show the *construction* is right, `generate_k()` takes its MAC as a parameter and the test suite instantiates it with HMAC-SHA256 and checks against RFC 6979’s own published k values.

Signature malleability. We normalise to low- s . Without it, $(r, n-s)$ is an equally valid signature, so the bytes do not identify a signature uniquely. This does not affect non-repudiation — both forms trace to the same signer — but we preferred not to build anything on a value an attacker can flip.

3.4 Trusting a public key (§7.4)

Chosen: a certificate authority we implemented, plus fingerprint comparison as a second layer.

This is the problem the specification calls the hardest, and it is: if Layla asks the server for Omar’s key and the server substitutes its own, every later guarantee collapses. The server would decrypt everything, re-encrypt to Omar, and both would see a working system — correctly authenticating the attacker.

Rejected. Out-of-band fingerprint comparison alone is sound but puts work on users every single time, and nobody does it. **Trust on first use** is easy and detects later changes, but is defenceless exactly when it matters — first contact, the only moment the server needs to lie. **A short authentication string** derived from both keys is strong but needs an interactive session between the two users, which our store-and-forward model does not have.

Chosen, and why. A CA moves the out-of-band step from one-per-pair-of-users to one-in-total: the CA’s public key ships with the client, and every user key is then vouched for by a signature under it. We kept fingerprint comparison as an independent second layer for users who do not want to rely on the CA alone.

What happens on first interaction. Layla asks the server for Omar’s certificate. She verifies the CA signature over it, that it is in date, and that the name inside the signed region is the name she asked for. All three checks are in `ca.verify_certificate()`, and there is no code path in the client that parses a certificate without calling it.

What it assumes. The CA is a trusted third party — we have moved the problem rather than eliminated it. Ours signs whatever username is requested, first come first served; it has no way to check that the person registering as `layla` is Layla. In a real deployment that check is the CA’s entire job and it happens out of band. §5.1 carries this as a limitation.

A vulnerability we found by attacking ourselves. Our first version accepted any Python identifier as a username. Python identifiers may contain Unicode, so `omar` — with a Cyrillic *o* — is a legal, distinct username that renders identically to `omar`. An attacker registers it, the CA signs it without complaint, and Layla reading her own screen cannot tell which she is sharing with. Every cryptographic guarantee would hold perfectly while the document went to the wrong person. We now restrict usernames to lowercase ASCII letters, digits, underscore and hyphen. This closes the Unicode form and **not** the ASCII form (`rmark` vs `mark`); see §5.1.

3.5 Signatures versus MACs (§7.5)

Mechanism	Where	Why that tool
GCM tag (a MAC)	document contents and metadata; keystore; keyring; share record	both parties are the same party — a user protecting their own data against a server that might alter it. No third party needs convincing.
ECDSA signature	the manifest inside every share	the verifier must not be able to forge what they verify

Which can satisfy non-repudiation, and why the other cannot. Only the signature, and the reason is structural rather than a matter of tag length. A MAC key is shared: if Omar can verify a tag, Omar could equally have produced it. His evidence proves only that *one of the two of them* made it, and he is not a neutral party. A 512-bit MAC tag changes nothing about this. A signature is verified with a public key that verifies but cannot sign, so the only party who could have produced it is the holder of the private key.

Sign-then-encrypt, and the danger it carries. We sign the manifest, then encrypt `manifest || signature || document key`. The known hazard with this order is *surreptitious forwarding*: Omar decrypts, re-encrypts the same signed manifest to Mallory, and Mallory’s client sees a valid Layla signature and concludes Layla wrote to her. We defeat it by putting the intended recipient’s identity **inside the signed region**. Mallory’s client checks that the name inside the signature is her own, finds `omar`, and refuses.

Why not encrypt-then-sign. The signature would then be over the ciphertext and visible to the server, telling it who sent what — and an attacker could strip the signature and re-sign the same ciphertext, claiming authorship of a document they cannot read.

Why the manifest hashes the plaintext, not the ciphertext. Layla signs what she means, not how it was packaged. Omar decrypts, hashes what he got, and compares. A third party can check the same thing holding no secret at all.

3.6 Architecture and formats (§7.6)

Chosen: client and server communicating through a shared directory. The specification says this is entirely adequate; we chose it positively. It gives the attacker the strongest position the threat model allows — they open the file and edit it, with no races to win — so testing against it tests the real threat. It makes the server’s storage inspectable during the demonstration (requirement 3 becomes cat). And sockets would have cost a day of connection handling and bought no security property.

What a client reports when verification fails. One error type, one message. A client that distinguished “bad tag” from “bad signature” from “stale” would let an attacker probe which modifications were noticed; a server that distinguished “wrong password” from “no such user” would leak the user list. The specific reason is available locally through a `detail` attribute for the user’s own actions — they already hold the key material involved — and the demonstration uses it to show which check fired. It never crosses the network.

4. Security argument

Property	Mechanism
Confidentiality	AES-128-GCM under a fresh per-document key. The key reaches the recipient only inside a payload encrypted under a key derived by ECDH from two private keys the server does not hold. The server never receives any key that opens a document.
Integrity	The 128-bit GCM tag over ciphertext and metadata together, checked before any plaintext is released. For shared documents, additionally <code>BLAKE2b(plaintext)</code> inside the sender’s signature.
Data-origin authentication	ECDSA over the manifest, verified against the signing key in the sender’s CA-issued certificate. The certificate binds the name to the key; the CA’s public key reached the client out of band.
Non-repudiation	The same signature, plus the fact that the recipient <i>keeps</i> the manifest, the signature and the certificate. <code>verify_receipt()</code> checks the chain holding only the CA public key — no secret, and no trust in the holder.
Credential protection	Argon2id with a per-user salt, at a work factor quantified in Appendix B. The server stores <code>BLAKE2b(auth_token)</code> , two hashes away from the password, and the expensive one is not invertible without guessing.
Freshness	Two mechanisms. A signed timestamp with a 300-second window bounds how long a captured object is useful. A record of every accepted manifest identifier, kept inside the encrypted keyring so it follows the user between machines, refuses replays <i>inside</i> the window, which a timestamp alone cannot do.
Metadata binding	The metadata header is the entire AAD of the document’s GCM encryption — readable by the server, unalterable by it. For shared documents, <code>BLAKE2b(metadata header)</code> is additionally inside the sender’s signature. The document id and nonce are inside the AAD, so a ciphertext cannot be moved under a different header.

How they fit together. The specification warns that a design satisfying each property in isolation can still fail when combined, the classic example being correct encryption and correct authentication composed in the wrong order. Three composition points matter here:

1. *Within a document*, GCM composes encryption and authentication in the encrypt-then-MAC order by construction, and we never touch that order.
2. *Within a share*, we compose a signature with an encryption in the sign-then-encrypt order, which is the order that leaks less and prevents authorship-stripping — at the cost of surreptitious forwarding, which we close by binding the recipient’s identity into the signed region.
3. *Across objects*, the manifest binds the document to the share: the same signed statement commits to the plaintext hash, the metadata hash, both identities, the document id and the time. An attacker who mixes pieces from two valid exchanges produces an object where one of those bindings fails.

What an attacker learns from a full database dump. The user list; who shared what with whom and when; every file name, size and type; and the ciphertexts. Not: any document's contents, any private key, or any password except by guessing at the cost in Appendix B.

4.1 Compliance with the from-scratch rule (§9)

Implemented by us, with no cryptographic library: BLAKE2b, Argon2id (all three variants), AES-128, CTR, GHASH, AES-GCM, P-256 field and curve arithmetic, ECDH, ECDSA, RFC 6979 deterministic nonces, and the key-derivation hierarchy.

That is every cryptographic component the system uses — 100%, not 70%.

From libraries: Python's arbitrary-precision integers, `os.urandom`, and the file, argument-parsing and interface code. The rules classify these as non-cryptographic.

Used in the test suite only, for cross-checking: `hashlib`, `hmac`, `cryptography`, `argon2-cffi`. No production code path imports any of them; the tests skip cleanly if they are absent, and the published vectors remain the primary evidence either way.

5. Limitations

An honest account of what this design does not stop.

5.1 The certificate authority is a trusted third party

Our CA signs whatever username is requested, first come first served. It cannot check that the person registering as `layla` is Layla — that check is what a real CA is for and it happens out of band. A compromised CA can bind any name to any key and break everything downstream.

Restricting usernames to lowercase ASCII closed the Unicode homograph attack (§3.4). It does not close the ASCII look-alikes: `rnark` against `mark`, `l` against `1`. Fingerprint comparison is what a careful user can do about it, and `cli.py` fingerprint exists for that.

What it would take to fix: identity verification at registration, and a confusable-name check against already-registered usernames.

5.2 No forward secrecy

Both key pairs are long-term, so this is static-static ECDH. An attacker who records the server's storage today and later compromises Omar's private key can decrypt every document ever shared with him.

What it would take: an ephemeral key pair per share on the sender's side, its public half published alongside the wrapped key. One extra scalar multiplication and 65 bytes per share. We did not implement it.

5.3 No revocation

Certificates carry an expiry but there is no revocation list. A user whose private key is stolen cannot invalidate their certificate before it expires, and we have no mechanism for re-keying an identity.

5.4 Metadata is visible to the server

File names, sizes, types, timestamps, owners, and the full share graph are in the clear. Section 4 places traffic analysis out of scope, but we should be clear that the exposure here is larger than "timing and sizes": the file name is plaintext because the server must list it. Encrypting names under a per-user key and giving the server only an opaque identifier would fix the names; the share graph is harder and would need something like a private information retrieval scheme.

5.5 A guessable password is still guessable

Appendix B shows that a password from the 10,000 most common falls in under six seconds even at the production work factor. No work factor fixes this — it multiplies the cost of a search, and multiplying a small number leaves a small number. A dictionary check at sign-up would address it; we have not implemented one.

The demo profile is visibly inadequate against a real attacker and exists only for the live demonstration.

5.6 The implementation is not side-channel hardened

`scalar_mult` uses double-and-add-always, removing the blatant key-dependent branch, and tag comparison is constant-time. But Python's arbitrary-precision integer arithmetic is not constant-time — operations on small values are genuinely faster — so we do not claim side-channel resistance. Our threat model places the user's machine outside the attacker's reach, which is why we accept this; in any other threat model it would be a serious gap.

Our Argon2id is ~300× slower than compiled implementations, discussed at length in §3.1.

5.7 The keyring is rewritten whole

Every upload and every accepted share rewrites the user's entire keyring. Two clients logged in as the same user simultaneously can lose one another's writes. The accepted-manifest list also grows without bound. Neither is a confidentiality problem, but both are real defects at scale.

5.8 No protection against a server that deletes

Our threat model says the server neither deletes nor fabricates data. A server that simply withholds a share, or serves an old copy of the keyring, is not detected. Detecting that needs something like a signed, monotonically increasing version number per user.

5.9 Sessions

Session tokens are bearer tokens with a one-hour lifetime. The server stores only a hash of each, so a stolen database yields no usable sessions, but anyone who observes a token in transit can use it. In our simulated network the “transit” is a function call; over a real network this would need TLS or a challenge-response scheme.

6. AI usage statement

We used Claude as the only AI assistant on this project.

The boundary we applied. The assistant was used for how our stored objects and protocol messages are encoded and decoded, and how those records are passed between client and server through the shared directory of Section 7.6.

(a) Debugging and syntax. We used the assistant to read Python error messages, interpret failing tests and look up standard-library syntax while implementing the primitives of Section 7. It helped us locate failures; the corrections were written and understood by us.

(b) Boilerplate and utility code. The byte-level format of our stored objects and protocol messages was decided by our team as part of Section 7.6 and is set out in the format tables of this report. Working from those fixed tables, the assistant wrote the serialization and deserialization code in `formats.py` that implements them, and the shared-directory transport code in `client.py` and `server.py` that writes and reads those records. The project specification's Section 14 lists file handling and interface scaffolding as permitted boilerplate, and none of this code makes a cryptographic or protocol decision. The sequence of messages in each operation, the conditions under which a client rejects an object, and the sign-then-encrypt ordering of Section 7.5 were written by us.

(c) Test coverage. Our team wrote the initial correctness tests for the primitives (`aes_gcm_test.py`, `argon2_test.py`, `blake2b_test.py`, `kdf_test.py`, `P256_test.py`). We then used the assistant to add further test cases covering inputs our own tests had not exercised. We read and ran each of those tests ourselves.

(d) Attacks. The programs in `attacks/` — `attack_mitm.py`, `attack_tamper.py`, `attack_replay.py`, `attack_weak_nonce.py` and `attack_offline_guessing.py` — were written by our team. We used the assistant to help identify which attacks were worth implementing against our design; selecting, implementing, running and interpreting them was our work.

(e) This statement. The facts stated in this section were supplied by our team; the wording was drafted with the assistant's help and reviewed by us.

Produced entirely without AI assistance. Every primitive listed as implemented from scratch in Section 7 was designed and written by our team: BLAKE2b, Argon2id, AES-128, CTR, GHASH, GCM, P-256 arithmetic, ECDH, ECDSA and the KDF (`blake2b.py`, `argon2.py`, `aes.py`, `gcm.py`, `P256.py`, `ECDH.py`, `ECDSA.py`, `kdf.py`), including the RFC 6979 deterministic nonce construction and the BLAKE2b domain-separation scheme of Section 7. So were the certificate authority of Section 7.4 (`ca.py`), the protocol logic — the sign-up, login, upload, share and retrieve exchanges, the per-document key arrangement of Section 7.2, the signed manifest and sign-then-encrypt ordering of Section 7.5, and the freshness check — and every decision in Section 7 together with the alternatives we rejected and the reasoning behind each. The format tables and all text in this report are our own.

7. Contribution statement

Member	Contribution
Abdallah Hattab (1230588)	P256 → ECDA → ECDSA end to end test demo client format
Abdallah Murra (1230534)	Blake2b → argon2id → KDF Attacks ca cli
Osama Aabed (1201164)	AES → CTR → GHASH → GCM server storage params

Appendix A — Sources consulted

Per §17, every source is documented. We consulted these specifications and educational materials; no implementation was copied from any of them.

- RFC 7693 — *The BLAKE2 Cryptographic Hash and MAC*, Saarinen & Aumasson, 2015. Algorithm and test vectors.
- RFC 9106 — *Argon2 Memory-Hard Function*, Biryukov, Dinu, Khovratovich & Josefsson, 2021. Algorithm and test vectors.
- FIPS 197 — *Advanced Encryption Standard*, NIST, 2001. AES and test vectors.
- NIST SP 800-38D — *Galois/Counter Mode (GCM) and GMAC*, 2007.
- McGrew & Viega — *The Galois/Counter Mode of Operation*, 2004. Test cases.
- FIPS 186-4 — *Digital Signature Standard*, NIST, 2013. P-256 parameters, ECDSA.
- SEC 1 / SEC 2 — *Elliptic Curve Cryptography and Recommended Domain Parameters*, Certicom. Point encoding.
- NIST SP 800-56A — Public key validation criteria.
- RFC 6979 — *Deterministic Usage of DSA and ECDSA*, Pornin, 2013. Algorithm and test vectors.
- RFC 3526 — MODP groups, for the finite-field comparison in §3.3.
- NIST SP 800-57 Part 1 — Comparable security levels.
- A. Joux — *Authentication Failures in NIST version of GCM*, 2006. Cited in §3.2 for the consequences of nonce reuse.
- Explicit formulas database (hyperelliptic.org/EFD) — the dbl-2001-b Jacobian doubling formula for $a = -3$.
- Course materials, ENCS4320 Lectures 01–06 (Dr. Ahmed Shawahna).

Appendix B — Offline attack arithmetic

Produced by `attacks/attack_offline_guessing.py`. Assumptions are printed with the output so they can be disputed: GPU memory bandwidth 1 TB/s at 35% achieved efficiency, 24 GB VRAM, bare SHA-256 at 20 GH/s, expected work half the space.

Argon2id is limited by memory traffic — one guess moves roughly 2-tm bytes — so the attacker’s rate is bandwidth divided by that.

Measured on our machine, one CPU core running the compiled reference implementation achieves **11 guesses per second** at the production profile and **764 per second** at the demo profile. The GPU figures below are derived from the parameters rather than measured, since we have no GPU to measure on; the script prints its assumptions alongside its answer so they can be disputed.

Production profile (m = 64 MiB, t = 3, p = 4). 403 MB moved per guess; one GPU achieves ≈870 guesses/second.

Password space	one GPU	1,000 GPUs
10,000 most common	5.8 seconds	instant
8 chars, lowercase (26^8)	4 years	1.4 days
8 chars, mixed case + digits (62^8)	3,980 years	4 years
4 random words from 7,776	66,643 years	67 years

Demo profile (m = 1 MiB, t = 1, p = 1). 2.1 MB per guess; ≈167,000 guesses/second on one GPU.

Password space	one GPU	1,000 GPUs
10,000 most common	instant	instant
8 chars, lowercase	7.2 days	10.4 minutes
8 chars, mixed case + digits	21 years	7.6 days
4 random words	347 years	127 days

What the work factor buys. For an 8-character mixed-case-plus-digits password on one GPU: a single bare SHA-256 falls in **1.5 hours**; Argon2id at the production profile takes **3,980 years** — a factor of 2.3×10^7 .

What it does not buy. A password from the 10,000 most common falls in 5.8 seconds at the production work factor. See §5.5.